

```
# Cell 1 – imports
import nltk; nltk.download('punkt', quiet=True)
import os, time, math, random
import numpy as np, matplotlib.pyplot as plt
import torch, torch.nn as nn, torch.optim as optim
from nltk.translate.bleu_score import sentence_bleu, SmoothingFunction
SEED = 42
random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)
```

Using device: cuda

```
# Cell 2 – upload dataset
if not os.path.exists("vast_english_french.txt"):
    from google.colab import files
    print("Please choose vast_english_french.txt ..."); files.upload()
print("Dataset present:", os.path.exists("vast_english_french.txt"))
```

Dataset present: True

```

# Cell 3 – vocab, split, tensors
PAD_token, SOS_token, EOS_token, UNK_token = 0, 1, 2, 3
class Vocabulary:
    def __init__(self):
        self.word2index = {"<PAD>":0,"<SOS>":1,"<EOS>":2,"<UNK>":3}
        self.index2word = {0:"<PAD>",1:"<SOS>",2:"<EOS>",3:"<UNK>"};
    def add_sentence(self, s):
        for w in s.split(' '): self.add_word(w)
    def add_word(self, w):
        if w not in self.word2index:
            self.word2index[w]=self.n_words; self.index2word[self.n_w
    def encode(self, s, max_len):
        ids=[SOS_token]+[self.word2index.get(w,UNK_token) for w in s.]
        ids=ids[:max_len]; return ids+[PAD_token]*(max_len-len(ids))
def load_pairs(path):
    pairs=[]
    with open(path, encoding="utf-8") as f:
        for line in f:
            line=line.rstrip("\n")
            if not line.strip(): continue
            p=line.split("\t")
            if len(p)==2 and p[0].strip() and p[1].strip(): pairs.append(p)
    return pairs
all_pairs=load_pairs("vast_english_french.txt"); print("Total pairs:",len(all_pairs))
rng=random.Random(SEED); idx=list(range(len(all_pairs))); rng.shuffle(idx)
split=int(0.8*len(all_pairs))
train_pairs=[all_pairs[i] for i in idx[:split]]; val_pairs=[all_pairs[i] for i in idx[split:]]
print(f"Train: {len(train_pairs)} | Val: {len(val_pairs)}")
eng_vocab, fr_vocab = Vocabulary(), Vocabulary()
for e,f in train_pairs: eng_vocab.add_sentence(e); fr_vocab.add_sentence(f)
print(f"EN vocab: {eng_vocab.n_words} | FR vocab: {fr_vocab.n_words}")
MAX_LEN=max(max(len(e.split()) for e,_ in all_pairs), max(len(f.split()) for _,f in all_pairs))
def make_tensors(pairs, sv, tv, ml):
    src=torch.tensor([sv.encode(e,ml) for e,_ in pairs],dtype=torch.long)
    tgt=torch.tensor([tv.encode(f,ml) for _,f in pairs],dtype=torch.long)
    train_src,train_tgt=make_tensors(train_pairs,eng_vocab,fr_vocab,MAX_LEN)
    val_src,val_tgt=make_tensors(val_pairs,eng_vocab,fr_vocab,MAX_LEN)

```

```

Total pairs: 555
Train: 444 | Val: 111
EN vocab: 903 | FR vocab: 1004

```

```
# Cell 4 – models
class EncoderGRU(nn.Module):
    def __init__(self, input_size, hidden_size, dropout=0.3):
        super().__init__(); self.hidden_size=hidden_size
        self.embedding=nn.Embedding(input_size,hidden_size,padding_id=0)
        self.dropout=nn.Dropout(dropout); self.gru=nn.GRU(hidden_size,hidden_size)
    def forward(self, x, h):
        emb=self.dropout(self.embedding(x)).unsqueeze(1); out,h=self.gru(emb,h)
    def initHidden(self): return torch.zeros(1,1,self.hidden_size,device=device)

class DecoderGRU(nn.Module):
    def __init__(self, hidden_size, output_size, dropout=0.3):
        super().__init__()
        self.embedding=nn.Embedding(output_size,hidden_size,padding_id=0)
        self.dropout=nn.Dropout(dropout); self.gru=nn.GRU(hidden_size,hidden_size)
        self.out=nn.Linear(hidden_size,output_size); self.log_softmax=nn.LogSoftmax(-1)
    def forward(self, x, h):
        emb=self.dropout(self.embedding(x)).view(1,1,-1); o,h=self.gru(emb,h)
        return self.log_softmax(self.out(o[0])), h
```

```
# Cell 5 – loss + epoch runner (teacher forcing in both modes)
def seq_loss(enc, dec, src, tgt, crit):
    h=enc.initHidden(); sl=(src!=PAD_token).sum().item(); _,h=enc(src[:sl])
    di=torch.tensor([[SOS_token]],device=device); dh=h; loss,n=0.0,0
    tl=(tgt!=PAD_token).sum().item()
    for t in tgt[1:tl]:
        o,dh=dec(di,dh); loss=loss+crit(o,t.unsqueeze(0)); n+=1; di=t
    return loss, max(n,1)

def run_epoch(enc, dec, st, tt, crit, eo=None, do=None, train=True):
    enc.train(train); dec.train(train); total,count=0.0,0
    order=list(range(st.size(0)))
    if train: random.shuffle(order)
    ctx=torch.enable_grad() if train else torch.no_grad()
    for i in order:
        src,tgt=st[i].to(device),tt[i].to(device)
        if train: eo.zero_grad(); do.zero_grad()
        with ctx: loss,n=seq_loss(enc,dec,src,tgt,crit)
        if train:
            loss.backward(); nn.utils.clip_grad_norm_(enc.parameters(),5.0)
            nn.utils.clip_grad_norm_(dec.parameters(),5.0); eo.step()
        total+=loss.item(); count+=n
    return total/count
```

```
# Cell 6 – train, keep best-BLEU checkpoint (dropout eased to 0.1)
```

```

HIDDEN, LR, EPOCHS, PATIENCE=256, 0.001, 60, 12
encoder=EncoderGRU(eng_vocab.n_words, HIDDEN, dropout=0.1).to(device)
decoder=DecoderGRU(HIDDEN, fr_vocab.n_words, dropout=0.1).to(device)
criterion=nn.NLLLoss()
eo=optim.Adam(encoder.parameters(), lr=LR); do=optim.Adam(decoder.parameters())

def _greedy(enc, dec, src, max_len=MAX_LEN):
    enc.eval(); dec.eval()
    with torch.no_grad():
        h=enc.initHidden(); sl=(src!=PAD_token).sum().item(); _, h=enc(s
        di=torch.tensor([[SOS_token]], device=device); dh=h; out=[]
        for _ in range(max_len):
            o, dh=dec(di, dh); _, topi=o.topk(1); ix=topi.item()
            if ix==EOS_token: break
            if ix!=PAD_token: out.append(ix)
            di=topi.detach().view(1,1)
    return out

def _val_bleu():
    sm=SmoothingFunction().method1; tot=0.0
    for i, (eng, fr) in enumerate(val_pairs):
        pred=[fr_vocab.index2word.get(k, "<UNK>") for k in _greedy(encoder, fr)]
        ref=fr.split(' '); tot+=sentence_bleu([ref], pred, smoothing_function=sm)
    return tot/len(val_pairs)

train_losses, val_losses=[], []; best_bleu, best_state, bad=-1.0, None, 0; start=time.time()
for epoch in range(EPOCHS):
    tr=run_epoch(encoder, decoder, train_src, train_tgt, criterion, eo, do, train=True)
    va=run_epoch(encoder, decoder, val_src, val_tgt, criterion, train=False)
    vb=_val_bleu()
    train_losses.append(tr); val_losses.append(va)
    if vb>best_bleu:
        best_bleu, bad=vb, 0
        best_state=({k:v.cpu().clone() for k,v in encoder.state_dict().items(): v!=None},
                    {k:v.cpu().clone() for k,v in decoder.state_dict().items(): v!=None})
    else: bad+=1
    if (epoch+1)%5==0 or epoch==0: print(f"Epoch {epoch+1:02d}/{EPOCHS} | train loss: {tr:.4f} | val loss: {va:.4f} | val BLEU: {vb:.4f}")
    if bad>=PATIENCE: print(f"Early stop at epoch {epoch+1}"); break
print(f"Time: {time.time()-start:.1f}s | best val BLEU: {best_bleu:.4f}")
encoder.load_state_dict(best_state[0]); decoder.load_state_dict(best_state[1])

```

```

Epoch 01/60 train=5.2074 val=4.5906 valBLEU=0.0527 (bestBLEU=0.0527)
Epoch 05/60 train=0.7611 val=4.7941 valBLEU=0.0941 (bestBLEU=0.0941)
Epoch 10/60 train=0.0886 val=5.4027 valBLEU=0.1032 (bestBLEU=0.1032)
Epoch 15/60 train=0.0233 val=5.7002 valBLEU=0.1091 (bestBLEU=0.1091)
Epoch 20/60 train=0.0056 val=6.1331 valBLEU=0.1102 (bestBLEU=0.1138)
Epoch 25/60 train=0.0415 val=6.4127 valBLEU=0.1110 (bestBLEU=0.1138)

```

```

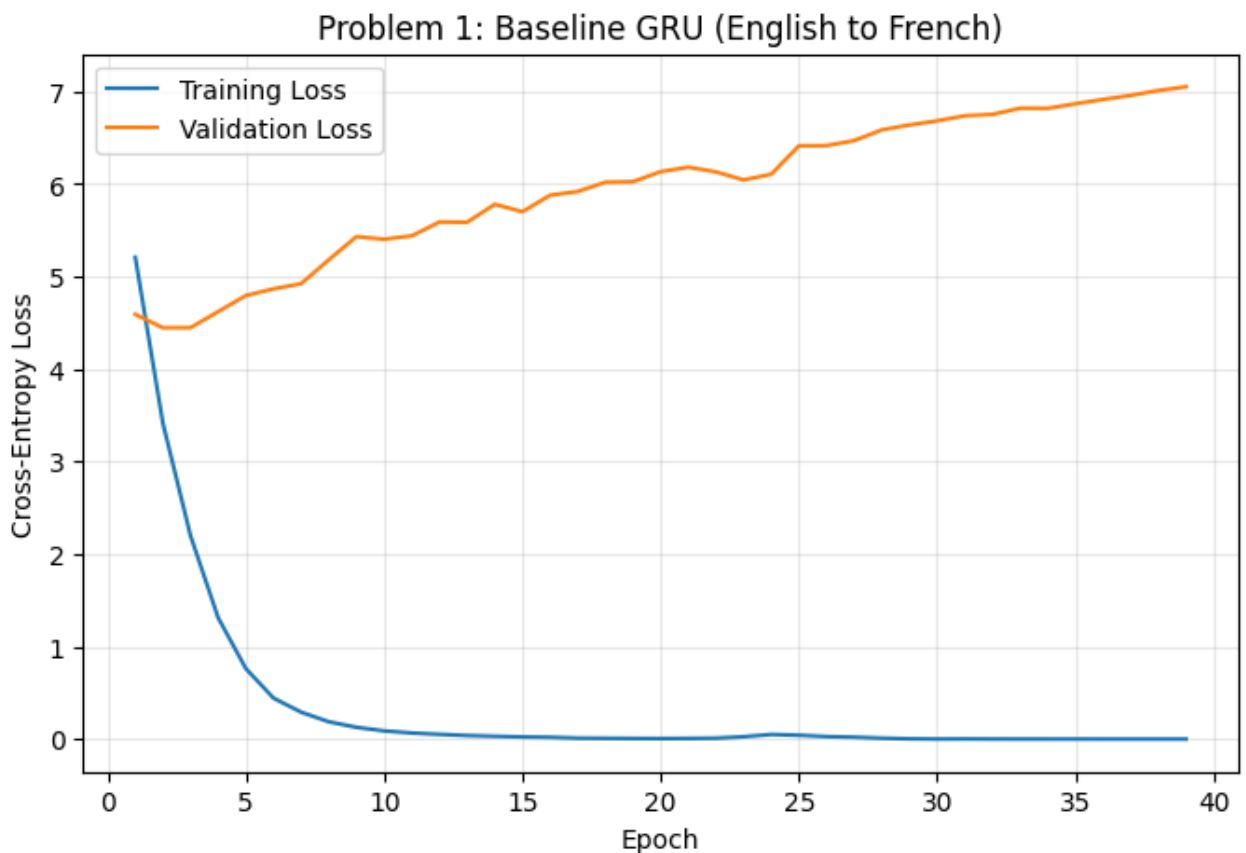
Epoch 30/60 train=0.0007 val=6.6823 valBLEU=0.1104 (bestBLEU=0.1152)
Epoch 35/60 train=0.0002 val=6.8670 valBLEU=0.1111 (bestBLEU=0.1152)
Early stop at epoch 39
Time: 252.2s | best val BLEU: 0.1152
DecoderGRU(
  (embedding): Embedding(1004, 256, padding_idx=0)
  (dropout): Dropout(p=0.1, inplace=False)
  (gru): GRU(256, 256)
  (out): Linear(in_features=256, out_features=1004, bias=True)
  (log_softmax): LogSoftmax(dim=1)
)

```

```

# Cell 7 - loss curves
plt.figure(figsize=(8,5))
plt.plot(range(1,len(train_losses)+1),train_losses,label="Training Loss")
plt.plot(range(1,len(val_losses)+1),val_losses,label="Validation Loss")
plt.title("Problem 1: Baseline GRU (English to French)")
plt.xlabel("Epoch"); plt.ylabel("Cross-Entropy Loss"); plt.legend(); pl

```



```

# Cell 8 – metrics + samples
def translate(enc, dec, src, max_len=MAX_LEN):
    enc.eval(); dec.eval()
    with torch.no_grad():
        h=enc.initHidden(); sl=(src!=PAD_token).sum().item(); _,h=enc
        di=torch.tensor([[SOS_token]],device=device); dh=h; out=[]
        for _ in range(max_len):
            o,dh=dec(di,dh); _,topi=o.topk(1); ix=topi.item()
            if ix==EOS_token: break
            if ix!=PAD_token: out.append(ix)
            di=topi.detach().view(1,1)
    return out
def ids_to_words(ids,v): return [v.index2word.get(i,"<UNK>") for i in
def evaluate(enc,dec,pairs,st,tv,n_show=5):
    sm=SmoothingFunction().method1; ex,tb,shown=0,0.0,[]
    for i,(eng,fr) in enumerate(pairs):
        pred=ids_to_words(translate(enc,dec,st[i]),tv); ref=fr.split(
        b=sentence_bleu([ref],pred,smoothing_function=sm) if pred else
        if i<n_show: shown.append((eng,fr,' '.join(pred),ok,b))
    n=len(pairs); return ex/n,tb/n,shown
acc,bleu,samples=evaluate(encoder,decoder,val_pairs,val_src,fr_vocab)
print(f"Traditional Sequence Accuracy (exact match): {acc*100:.2f}%")
print(f"Average Validation BLEU-4: {bleu:.4f}\n")
print(f"{'English':<38}{'French (ref)':<34}{'Prediction':<34}{'Match'
for eng,fr,pred,ok,b in samples:
    print(f"{'eng[:37]:<38}{'fr[:33]:<34}{'pred[:33]:<34}{'('PASS' if ok

```

Traditional Sequence Accuracy (exact match): 0.00%
Average Validation BLEU-4: 0.1152

English

French (ref)

They feed the pigeons in the square	Ils nourrissent les pigeons sur
She practices yoga every morning	Elle pratique le yoga tous les ma
I enjoy walking in the snow	J'aime marcher dans la neige
She loves to wear modern jackets	Elle adore porter des vestes mode
The city bus arrives precisely at fiv	Le bus de la ville arrive précise

